

High-Performance Implementation of Multigrid Solver for Lattice QCD

10 分钟汇报讲稿

张鑫

中国科学院近代物理研究所

2026 年 10 月 11 日

演讲总时长：10 分钟。横版汇报稿共 17 页，建议平均每页 35 秒；公式和算法页约 35–40 秒，性能图页约 30–35 秒，封面和致谢页各约 10 秒。以下文字为逐页口播稿，不是论文摘要；遇到时间不足时，优先保留每页加粗的判断句和数字。

页码	主题	建议时长	必须保留的信息
1	标题与范围	15 s	只汇报 MultiGrid，数据基线为 test27
2–4	Wilson/Clover/Dslash/Schur	100 s	传播子、Clover 块、奇偶消元
5–8	Multigrid 与算法	140 s	粗空间、Galerkin、Bi-CGSTab、FGMRES
9	CUDA-C++ 优化	40 s	细层带宽、粗层同步、通信和显存
10–14	QUDA 对照与结论	175 s	66/132、35/44、21/22、容量边界
15–16	复现与先前成果	45 s	最小闸门、508 报告的 64→1024 扩展
17	致谢	10 s	一个结论和一个行动项

逐页讲稿

第 1 页 | 标题与汇报范围（约 15 秒）

各位老师、各位同行，大家好。我汇报的题目是面向格点 QCD 传播子求解的高性能多重网格实现。这里重点讲 PyQCU 的 MultiGrid，尤其是 CUDA-C++ 核心；性能数据统一采用 `git tag test27`，不把历史标签或实验性加速比混入最终结论。

视觉提示：停在标题、项目和 `test27` 三个位置上，不展开项目历史。

第 2 页 | Wilson 与 Clover 算子（约 35 秒）

首先固定细网格算子。裸跃迁核 H 由四个正向和四个反向 neighbors 组成；Wilson 算子是 $(m_0 + 4)I - \frac{1}{2}H$ ，等价地写成 $(m_0 + 4)(I - \kappa H)$ ，其中 $\kappa = 1/(2m_0 + 8)$ 。Clover 项是在相同最近邻结构上增加一个 onsite 的 12×12 spin-color 块 $A_p = I + T_p$ 。需要特别注意，C++ 的原始 Dslash 入口返回 H ，不是完整算子，也不包含 $-\kappa$ 因子。

视觉提示：先指 *Wilson* 公式，再指 *Clover onsite block*，最后指 “*raw H*” 警示栏。

第 3 页 | 原始 Dslash (约 30 秒)

原始 Dslash 是矩阵自由的最近邻稀疏作用。每个格点读取八个邻居；对每个方向执行 $1 \pm \gamma_\mu$ 投影后，再做规范平移和 $SU(3)$ 矩阵乘法。因此它天然 *bandwidth-bound*。PyQCU 将四分量 *spin projection* 降到两个独立分量，并利用 $SU(3)$ 结构和时空维尾置布局减少访存。这里的核心不是“多写几个 kernel”，而是把 Dslash 从条件分支和随机访存中解耦，使其成为 GPU 擅长执行的连续数据流。

视觉提示：沿右侧五个流程节点快速移动，不逐行朗读入口表。

第 4 页 | 奇偶 Schur 消元 (约 35 秒)

为了减小外层 Krylov 维度，PyQCU 将场分成 even 和 odd 两个子空间。以奇数子格为例，完整块算子包含 A_e, A_o 和两个 hopping block。精确消去 even 分量后得到 $S_o = A_o - \kappa^2 H_{oe} A_e^{-1} H_{eo}$ 。求解 $S_o x_o = b_o^c$ 后，再用 $x_e = A_e^{-1}(b_e + \kappa H_{eo} x_o)$ 重建完整解。这是精确代数变换，最终正确性仍由完整 Wilson/Clover 算子的真残差决定。

视觉提示：按“分块—消元—求解—重建”四步讲，避免陷入矩阵推导。

第 5 页 | MultiGrid 的核心构造 (约 35 秒)

MultiGrid 的关键是把误差分解成高频和低频。平滑器压低高频误差，粗空间表示低频误差。对每一层，先通过 $A_l B_l \approx 0$ 生成近零模，再在 *aggregate* 内做局部正交化，得到延拓 P_l ，并令 $R_l = P_l^\dagger$ 。粗算子通过 Galerkin 投影 $A_{l+1} = R_l A_l P_l$ 生成。局部正交意味着限制过程不需要全局 Gram 求逆，同时投影后的 coarse operator 仍保持有限支撑。

视觉提示：指公式 $A_{l+1} = R_l A_l P_l$ 和右侧 V-cycle 流程。

第 6 页 | 预条件 Bi-CGStab (约 40 秒)

这一页给出粗层求解器的完整算法。初始化残差后，每轮先计算 ρ_{i-1} ，构造搜索方向 $p^{(i)}$ ；然后通过预条件器求 $\hat{p}$ ，应用原算子得到 $v^{(i)}$ ，再计算 α_i 和中间残差 s 。如果 s 已经满足收敛条件，就立即更新 x 并停止；否则继续求 $\hat{s}, t = A\hat{s}$ ，计算 $\omega_i = t^\dagger s / (t^\dagger t)$ ，完成最终的 r 和 x 更新。粗层实现还保留热启动，但接近 *breakdown* 时回退冷启动。

视觉提示：只用手指沿表逐行扫过，不朗读所有公式；强调 $M\hat{p} = p$ 和 ω 两项。

第 7 页 | PyQCU Strict 特化 (约 35 秒)

PyQCU Strict 将粗层分解放成 $D_l = X_l + H_l$ ，再做左预条件 $\hat{D}_l = X_l^{-1} D_l$ ，最后投影得到 $D_{l+1} = R_l \hat{D}_l P_l$ 。运行期不保存完整粗矩阵，而保存 onsite X, X^{-1} 、轴链接 Y 和预条件后的

$\hat{Y} = X^{-1}Y$ 。细层是 compact target-parity 问题，粗层始终保留 full geometry。这个顺序非常重要：必须先 onsite 层做 X^{-1} ，再做 P/R ，否则会改变算子语义。

视觉提示：强调公式顺序和右侧 full coarse 数据流。

第 8 页 | V-cycle 与 FGMRES (约 40 秒)

一次 MG 预条件器在内部执行 pre-smoothing、restriction、递归粗层求解、prolongation 和 post-smoothing。最粗层用 Bi-CGStab，中间层递归。外层不直接用 CG，而是使用 flexible right-preconditioned GMRES：每次 Arnoldi 列都允许使用不同的预条件器。右表给出 FGMRES 的完整循环，包括 modified Gram-Schmidt、Givens 更新和 restart 后的 full true residual 刷新。这里的迭代计数只记外层 FGMRES，不能把 smoother 和 coarse solver 的迭代相加。

视觉提示：先讲左侧递归，再讲右侧 FGMRES；最后指工作区公式。

第 9 页 | CUDA-C++ 优化 (约 40 秒)

CUDA-C++ 的优化分为四层。细层 kernel 处理 spin projection、 $SU(3)$ 重用和合并访存；粗层通过 device-side scalars、融合标量更新和 CUDA Graph 减少 launch 与 host synchronization；setup 使用分块/着色 Galerkin、显存预算、持久 hierarchy 和 SHA256 runtime cache；多 rank 通信使用 rank-local 粗 halo、非阻塞 vector halo 和远端 correction 融合。complex64 默认启用 overlap，complex128 因浮点累加顺序敏感默认关闭。

视觉提示：按四列从上到下讲，不展开 API 和数组长度。

第 10 页 | QUDA 对照口径 (约 30 秒)

性能对照严格采用 test27 的最终矩阵。请求矩阵是 72 个 combined 单元，精确完成 66 个 combined、132 条 side record；其余 6 个双 P100 c64 大格请求项因显存或 570 秒 setup 时限被显式替代。每侧执行 1 次 cold、2 次 warmup、5 次 steady，正式性能使用 trace-off。所有精确单元都通过 config、input bundle 和 full-operator 真残差门。

视觉提示：四个 KPI 数字只需读出 72、66、132、6；右下角专门说明“替代不是精确结果”。

第 11 页 | 总体分布 (约 35 秒)

定义 $R = t_{\text{QUDA}}/t_{\text{PyQCU}}$ ， $R > 1$ 表示 PyQCU 更快。66 个精确单元的整体中位比值是 1.025，说明总体只是接近持平。但如果只看 MultiGrid，MG-2/3 有 35/44 个单元优于 QUDA；trace-off MG 则有 21/22 个单元有利。相反，BiCGStab reference 是 0/22，QUDA 全部更快。因此我们的结论必须限定在 MG 场景，不能外推到全部求解器。

视觉提示：先指整体中位，再指 MG 和 BiCGStab 两组，不逐点读数据。

第 12 页 | 设备与精度分组 (约 35 秒)

设备趋势非常清楚。V100 上 MG-3 的优势最大：c64 和 c128 的 trace-off 分组中位比分别是 4.74 和 3.22。双 P100 仍然总体有利，但幅度较小，MG-2/3 都在 1.15 到 1.79 左右。图中同时显示极值：V100 c128 MG-3 最大单点达到 14.63，但也存在 0.54 的慢点。因此中位数表示趋势，极值提醒我们结果对格点和残差轨迹敏感。

视觉提示：指色块从 V100 到 P100 的变化，不使用“全面领先”。

第 13 页 | 阶段、容量、残差与显存 (约 40 秒)

trace-on 阶段分解显示，PyQCU 的 coarse solve 与 other 合计约占 95%；这说明进一步优化细层算术的收益有限，重点应转到粗层同步、残差检查和通信。容量方面，双 P100 c64 大格的三条路径分别约为 15.9、14.8 和 12.4 GB；CPU staging 虽降低显存，但 setup 仍超过 570 秒。因此 6 个请求项被替换为 16^4 ，且不进入原大格速度比。正确性方面，PyQCU 和 QUDA 的最大真残差分别为 7.40×10^{-7} 和 7.97×10^{-7} ，均在 5×10^{-6} 门限以内。

视觉提示：左侧指阶段占比，右侧指容量替代表，底部指残差和显存 KPI。

第 14 页 | 结论、边界与下一步 (约 35 秒)

当前结论可以压缩成三句话。第一，MG 专项优势明确，尤其 V100 上显著；第二，整体与 BiCGStab 仍暴露明显边界，trace-on 同步和 P100 大格容量也限制推广；第三，下一步优先做粗层同步、持久 request、device-aware 通信、setup 显存和更多平台的端到端验证。我们不宣称任意格点、任意层数或所有求解器全面领先。

视觉提示：三栏分别讲优势、不足、下一步，最后强调不可外推。

第 15 页 | 参考实现与复现 (约 20 秒)

参考实现包括 quantum-mg、QUDA 和 DD-alphaAMG。复现顺序是先构建，再运行 Strict fast gate，随后运行双 rank MPI probe，最后才启动完整矩阵。这样可以在较长 benchmark 之前先排除 ABI、生命周期、halo 和真残差问题。当前平台优先级仍是 CUDA 性能优化，然后是 DCU/CANN 真机适配和 TileLang 内核化。

视觉提示：只读命令类别，不逐条念完整路径。

第 16 页 | 508 报告的大规模成果 (约 25 秒)

这一页补充先前成果。张鑫 508 报告在 4 卡 DCU 环境中记录了从 64 到 1024 进程规模的 Wilson 与 Clover 大格测试。报告中的累计加速比在 1024 进程时分别为 3.12 和 7.19，说明早期 DCU/CUDA 路径已经具备正确性和大规模扩展趋势；右下图给出 Clover 与 Wilson 单次迭代耗时差随等效体积增长的数据。需要明确，这是早期成果，当时 MG 仍属前期验证，不能与 test27 的 Strict-MG 正式速度比混合。

视觉提示：先读三个 *KPI*，再指两条扩展曲线，最后指原始 *TEST9* 表。

第 17 页 | 致谢 (约 10 秒)

我的汇报到这里。核心结论是 MultiGrid 专项性能已经进入国际先进实现同量级，并在 V100 等场景形成清晰优势；下一步要把粗层同步、通信和 setup 显存做成可复验的持续收益。谢谢大家。

视觉提示：停在总结句，随后进入提问。

备用问答要点

- **为什么总中位只有 1.025 ?** 因为 66 个单元同时包含 BiCGStab reference；MG 单独是 35/44，trace-off MG 为 21/22。
- **为什么 trace-on 不能作为正式性能 ?** trace-on 插入同步和记录，改变运行时开销；它只用于逐层归因，正式比值使用 trace-off。
- **为什么 c128 overlap 默认关闭 ?** 改变浮点累加顺序后，大格三层曾由 32 次外层迭代退化到 451 次，因此必须 fail-safe。
- **508 报告是否证明当前 Strict-MG 千卡扩展 ?** 不直接证明。它证明早期 DCU 传播子求解的大规模扩展能力；当前 Strict-MG 的公开强证据仍是 V100 单卡、双 P100 与 1/2/4 rank preflight。
- **最大的下一步是什么 ?** 降低 coarse solve 与同步成本，同时提升 setup 的显存效率，使未完成的大格请求进入正式精确矩阵。